

Contents lists available at ScienceDirect

Journal of Automation and Intelligence

journal homepage: www.keaipublishing.com/en/journals/journal-of-automation-and-intelligence/

Research article

Distributed computations for large-scale networked systems using belief propagation[☆]Qianqian Cai^a, Zhaorong Zhang^{b,*}, Minyue Fu^c^a School of Automation, Guangdong University of Technology, Guangzhou 510006, China^b Department of Applied Mathematics, Hong Kong Polytechnic University, Hong Kong Special Administrative Region of China^c School of System Design and Intelligent Manufacturing, Southern University of Science and Technology, Shenzhen 518055, China

ARTICLE INFO

Keywords:

Distributed estimation
Distributed optimisation
Sensor fusion
Distributed algorithm

ABSTRACT

This paper introduces several related distributed algorithms, generalised from the celebrated belief propagation algorithm for statistical learning. These algorithms are suitable for a class of computational problems in large-scale networked systems, ranging from average consensus, sensor fusion, distributed estimation, distributed optimisation, distributed control, and distributed learning. By expressing the underlying computational problem as a sparse linear system, each algorithm operates at each node of the network graph and computes iteratively the desired solution. The behaviours of these algorithms are discussed in terms of the network graph topology and parameters of the corresponding computational problem. A number of examples are presented to illustrate their applications. Also introduced is a message-passing algorithm for distributed convex optimisation.

1. Introduction

This paper is concerned with distributed computation problems for large scale networked systems, as depicted in Fig. 1. Examples of such systems include sensor networks, smart grid, smart manufacturing systems, multi-agent systems, traffic networks, social networks, and so on. Such a system often involves a large number of nodes (or agents, or sub-systems) scattered in a large geographical area, linked via physical and communication interactions. Two predominant features of these systems are (1) sparsity of the network connections expressed by a sparse network graph and (2) limited computing and communication capabilities at each node. It is necessary to fuse all the available information available from the entire network to deliver the best global solutions or performances. Therefore, distributed computation is essential for dealing with such large scale networked systems. For a large networked system such as sensor networks [1–3], networked control systems [4–6], network-based state estimation [7–11], biological networks [12, 13], multi-agent systems [14–16], multi-agent optimisation [17–19], distributed learning [20–26] and so on, it is often desirable to use distributed algorithms.

In this paper, we give a comprehensive survey of a class of message-passing distributed solutions for a number of important network computation problems. These problems include weighted average consensus, weighted least squares estimation, posterior probability computation and sparse linear systems. In each of these problems, we present

a fully distributed algorithm and discuss their convergence properties. We also present examples to illustrate their performances.

The development of these algorithms and its theoretical analysis follow from the so-called Gaussian belief propagation (BP) algorithm which applies to Gauss–Markov random fields which involve a symmetric system matrix [21,23–25]. The Gaussian BP algorithm is a variant of the celebrated Pearl’s belief propagation algorithm [20], also known as message passing algorithm or sum–product algorithm in the literature [21]. The algorithm runs iteratively on each node of the graph associated with the system, and enjoys low complexities on local information exchange between neighbouring nodes, local computation and local storage. For an acyclic graph, the algorithm converges to the exact solution in d iterations, where d is the diameter of the graph. This is also the theoretical minimum number of iterations required to solve a linear system using a distributed algorithm. For a loop graph, the algorithm is guaranteed to converge to the exact solution asymptotically. We will also study a distributed nonlinear optimisation problem and introduce a message-passing algorithm (also a variant of the BP algorithm) and provide the convergence properties of the algorithm under certain pairwise separation and scaled diagonal dominance conditions.

The presented algorithms are collated and extended from the works of [8,21,23,27–32]. By presenting these results together, we get a unified view of these message-passing algorithms.

The rest of the paper is organised as follows. Section 2 formulates the four network computation problems and introduces the required

[☆] This work was supported in part by the of National Natural Science Foundation of China (U21A20476, U1911401, U22A20221, 62273100, 62073090).

* Corresponding author.

E-mail addresses: qianqian.cai@outlook.com (Q. Cai), zhaorong.zhang@polyu.edu.hk (Z. Zhang), fumy@sustech.edu.cn (M. Fu).

<https://doi.org/10.1016/j.jai.2023.06.003>

Received 26 April 2023; Received in revised form 11 June 2023; Accepted 25 June 2023

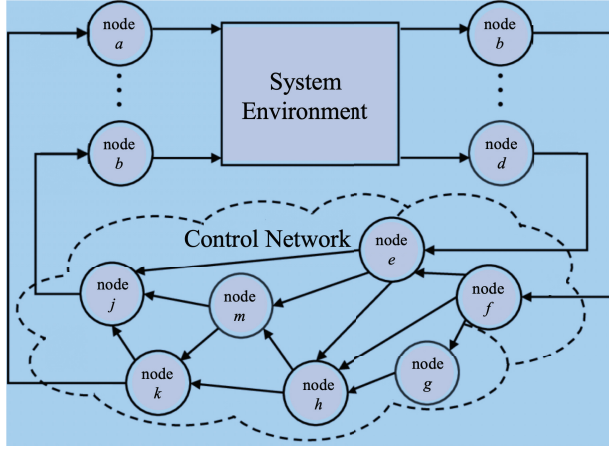


Fig. 1. Networked systems.

complexity constraints for distributed algorithms. Section 3 studies the Gaussian BP algorithm and its application for distributed computation of posterior probabilities. Section 4 gives a distributed algorithm for weighted average consensus. Section 5 describes a distributed algorithm for weighted least squares. Section 6 presents a distributed solver for linear systems. Section 7 studies the distributed convex optimisation problem. The convergence performances of the distributed algorithms are illustrated in simulation examples in Section 8. Conclusion remarks are reached in Section 9.

Notation. $\mathbb{R}^n$ denotes the n -dimensional Euclidean space. For $x \in \mathbb{R}^n$, x_i denotes its i th component, and $x = \text{col}\{x_1, x_2, \dots, x_n\}$. $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ denotes a graph with node set $\mathcal{V}$ and edge set $\mathcal{E}$. For any node $i \in \mathcal{V}$, $\mathcal{N}_i$ denotes the set of its neighbouring nodes, and $|\mathcal{N}_i|$ is its cardinality. The notation $m_{i \rightarrow j}$ means a message (i.e., data, information, or functions) transmitted from node i to node j . $\mathbb{E}$ denotes expectation. $\mathcal{N}(\mu, P)$ denotes the normal distribution with mean μ and covariance P . A probability density function $p(x) \propto f(x)$ means $p(x) = C f(x)$ for some constant $C > 0$. For a matrix $M \in \mathbb{R}^{n \times n}$, $\rho(M)$ denotes its spectral radius. An optimiser of a function $f(x)$ is denoted by x^* , and its estimate by $\hat{x}$. $O(N)$ is the big-O notation, meaning a function bounded by $aN + b$ for some non-negative constant a and b .

2. Problem formulations

In this section, we introduce several related distributed computation problems arising in estimation and sensor fusion in large-scale networked systems.

2.1. Weighted average consensus

Consider an undirected graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$. A graph is called *undirected* if information can flow in two ways between the two nodes on any edge. A graph is called *acyclic* if it is *connected* and has no loops, i.e., it is a *tree graph*. We are interested in a large graph with sparse connectivity, i.e., we assume that $|\mathcal{N}_i| \ll n$ for every $i \in \mathcal{V}$.

Let each node $i \in \mathcal{V}$ be associated with a variable (vector of a common dimension) u_i and a weight $w_i > 0$. Denote by $\bar{x}$ the weighted average of these u_i , i.e.,

$$\bar{x} = \frac{\sum_{i \in \mathcal{V}} w_i u_i}{\sum_{i \in \mathcal{V}} w_i}. \quad (1)$$

The problem of *distributed average consensus* [7] is to devise an iterative algorithm which runs on every node $i \in \mathcal{V}$ to compute an estimate of $\bar{x}$ such that after a certain number of iterations, each node's estimate will converge to $\bar{x}$. When $w_i = 1$ for all nodes, weighted average consensus becomes the standard (unweighted) average consensus.

2.2. Weighted least squares estimation

Consider a network of nodes $(1, 2, \dots, n)$ associated with a state vector $x = \text{col}\{x_1, x_2, \dots, x_n\} \in \mathbb{R}^n$ and a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ as described before. In the distributed weighted least squares problem we consider in this paper, two types of measurements for x are admitted: *self measurements* and *pairwise measurements*. A self measurement for node i is described by

$$z_i = A_i x_i + v_i, \quad (2)$$

where $A_i \neq 0$ and v_i is the measurement noise with independent normal distribution $\mathcal{N}(0, R_i)$, $R_i > 0$. Likewise, a pairwise measurement for and edge $(i, j) \in \mathcal{E}$ is described by

$$z_{ij} = B_{ij} x_i + B_{ji} x_j + v_{ij}, \quad (3)$$

where $[B_{ij} \ B_{ji}]$ has full row rank and v_{ij} is the measurement noise with independent normal distribution $\mathcal{N}(0, R_{ij})$, $R_{ij} > 0$. It is assumed that at least one node has a self measurement (i.e., not every node needs to have one self measurement), but every edge in $\mathcal{E}$ must have one pairwise measurement (otherwise, this edge can be removed from $\mathcal{E}$). It is assumed that A_i and R_i are known only to node i , and likewise, B_{ij} , B_{ji} and R_{ij} are known only to the edge (i, j) .

Denote $z = \text{col}\{\dots z_i \dots z_{ij} \dots\}$, $v = \text{col}\{\dots v_i \dots v_{ij} \dots\}$ and $R = \text{diag}\{\dots R_i \dots R_{ij} \dots\}$. Then the entire network's measurements can be expressed as

$$z = Hx + v, \quad (4)$$

where $v \sim \mathcal{N}(0, R)$ and

$$H = \begin{bmatrix} \dots & \dots & \dots \\ \dots & A_i & \dots \\ \dots & \dots & \dots \\ \dots & B_{ij} & \dots & B_{ji} & \dots \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix}$$

with the unspecified entries (...) in H equal to 0. It is clear that H is a sparse matrix.

The weighted least-squares problem [8] is to solve

$$x^* = \arg \min_x (z - Hx)^T R^{-1} (z - Hx). \quad (5)$$

2.3. Posterior probabilities in Gaussian graphical model

A fundamental problem in sensor fusing and statistical learning is how to compute posterior probabilities in a Gaussian graphical model using a distributed algorithm.

Consider a Gaussian graphical model with a joint probability density function for $x \in \mathbb{R}^n$:

$$p(x) \propto \exp\left\{-\frac{1}{2} x^T J x + h^T x\right\}, \quad (6)$$

where J is a symmetric positive definite matrix (called *information matrix*) and h is a vector (called *potential vector*). This is related to the standard Gaussian parameterisation in terms of the mean $\mu = \mathbb{E}\{x\} = J^{-1}h$ and covariance $P = \mathbb{E}\{(x - \mu)(x - \mu)^T\} = J^{-1}$. The associated Gaussian graphical model is the undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with $\mathcal{V} = \{1, 2, \dots, n\}$ and $\mathcal{E} = \{(i, j) | J_{ij} \neq 0, i, j \in \mathcal{V}\}$.

The problem of posterior probabilities is to compute, for each node i , the marginal mean μ_i and marginal variance P_{ii} , in a distributed way. In many applications, the computation of the means is far more important than that of the variances. Moreover, the matrix J is sparse, due to the sparsity of the Gaussian graphical model. From the above discussion, we see that this falls into a problem of sparse linear system:

$$J\mu = h. \quad (7)$$

2.4. Sparse linear systems

Consider a network of nodes $(1, 2, \dots, n)$ associated with a state vector $x = \text{col}\{x_1, x_2, \dots, x_n\} \in \mathbb{R}^n$. The information available at each node i is that x satisfies a linear system:

$$a_i^T x = b_i,$$

where $a_i = \text{col}\{a_{i1}, a_{i2}, \dots, a_{in}\} \in \mathbb{R}^n$ is a column vector and b_i is a scalar. Collectively, the common state satisfies

$$Ax = b \quad (8)$$

with $A = \text{col}\{a_1^T, a_2^T, \dots, a_n^T\}$ and $b = \text{col}\{b_1, b_2, \dots, b_n\}$. It is assumed throughout the paper that

(i) A is invertible; and

(ii) $a_{ii} \neq 0$ for all i .

Denote the solution of (8) by

$$x^* = A^{-1}b. \quad (9)$$

Define the *induced graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with $\mathcal{V} = \{1, 2, \dots, n\}$ and $\mathcal{E} = \{(i, j) | a_{ij} \neq 0 \text{ or } a_{ji} \neq 0\}$. We associate each node i with (a_i, b_i) , and each edge (i, j) with a_{ij} . Note in particular that $\mathcal{G}$ is *undirected*, i.e., $(i, j) \in \mathcal{E}$ if and only if $(j, i) \in \mathcal{E}$. For each node $i \in \mathcal{V}$, we define its *neighbouring set* as $\mathcal{N}_i = \{j | (i, j) \in \mathcal{E}\}$ and denote its *cardinality* by $|\mathcal{N}_i|$. We assume in the sequel that $|\mathcal{N}_i| \ll n$ for all $i \in \mathcal{V}$.

The *distributed linear system problem* we are interested in is to devise an iterative algorithm for each node $i \in \mathcal{V}$ to execute so that node i will be able to solve x_i . Note that in this problem formulation, node i is only interested in its local variable, and not interested in knowing the solution of x_j for any other node $j \in \mathcal{V}$, rather than every node trying to solve the entire x . This is motivated by the need of network scalability where asking every node to solve the entire x is often unnecessary and also infeasible when the network becomes large.

2.5. Complexity constraints

The distributed solutions for the problems above will be provided in the next four sections. In each of the problems, we want its distributed algorithm to be of low complexity and fast convergence. Certain *constraints* need to be imposed on the algorithm's complexities of communication, computation and storage to call it *distributed*. In our paper, these include:

- C1: Local information exchange: Each node i can exchange information with each $j \in \mathcal{N}_i$ only once per iteration.
- C2: Local computation: Each node i 's computational load should be at most $O(|\mathcal{N}_i|)$ per iteration.
- C3: Local storage: Each node i 's storage should be at most $O(|\mathcal{N}_i|)$ over all iterations.

Imposing these constraints is important to ensure that the produced distributed algorithms are scalable, i.e., the computational complexity per iteration at each node does not grow as the network size increases.

3. Gaussian BP for posterior probabilities

The Gaussian BP algorithm [21] is a variant of the celebrated belief propagation (BP) algorithm [20] for solving posterior probabilities in a Gaussian graphical model. It is fully distributed and very efficient. It aims to compute, for each node i , the marginal mean μ_i and marginal variance P_{ii} , in an iterative fashion. Rewriting $p(x)$ as

$$p(x) \propto \prod_{i \in \mathcal{V}} \psi_i(x_i) \prod_{(i, j) \in \mathcal{E}} \psi_{ij}(x_i, x_j) \quad (10)$$

with the node and edge potential functions

$$\psi_i(x_i) = \exp\{-\frac{1}{2}J_{ii}x_i^2 + h_i x_i\};$$

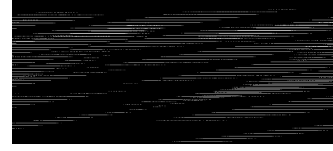


Fig. 2. Message passing in belief propagation [23].

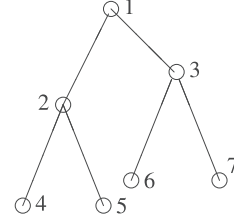


Fig. 3. An example of acyclic graph.

$$\psi_{ij}(x_i, x_j) = \exp\{-J_{ij}x_i x_j\} \quad (11)$$

The Gaussian BP algorithm iteratively passes messages between neighbouring nodes. That is, the message from node i to node $j \in \mathcal{N}_i$ in iteration k is given by

$$m_{i \rightarrow j}^{(k)} = \exp\{-\frac{1}{2}\Delta J_{i \rightarrow j}(k)x_j^2 + \Delta h_{i \rightarrow j}(k)x_j\}$$

or simply the pair $(\Delta J_{i \rightarrow j}(k), \Delta h_{i \rightarrow j}(k))$, which are computed by

$$\Delta J_{i \rightarrow j}(k) = -\frac{J_{ji}^2}{\hat{J}_{i \setminus j}(k)}; \quad \Delta h_{i \rightarrow j}(k) = -\frac{J_{ji}}{\hat{J}_{i \setminus j}(k)}\hat{h}_{i \setminus j}(k), \quad (12)$$

where

$$\begin{aligned} \hat{J}_{i \setminus j}(k) &= J_{ii} + \sum_{v \in \mathcal{N}_i \setminus j} \Delta J_{v \rightarrow i}(k-1); \\ \hat{h}_{i \setminus j}(k) &= h_i + \sum_{v \in \mathcal{N}_i \setminus j} \Delta h_{v \rightarrow i}(k-1) \end{aligned} \quad (13)$$

with $\Delta J_{i \rightarrow j}(-1) = 0$, $\Delta h_{i \rightarrow j}(-1) = 0$. The estimated marginal mean and variance for μ_i and P_{ii} are given by

$$\hat{\mu}_i(k) = \hat{J}_i^{-1}(k)\hat{h}_i(k); \quad \hat{P}_{ii}(k) = \hat{J}_i^{-1}(k), \quad (14)$$

respectively, with

$$\begin{aligned} \hat{J}_i(k) &= J_{ii} + \sum_{v \in \mathcal{N}_i} \Delta J_{v \rightarrow i}(k-1); \\ \hat{h}_i(k) &= h_i + \sum_{v \in \mathcal{N}_i} \Delta h_{v \rightarrow i}(k-1). \end{aligned} \quad (15)$$

The message passing process is illustrated in Fig. 2, where node i collects messages from its neighbouring nodes other than node j to form the message $m_{i \rightarrow j}$ and then transmits it to node j .

For an acyclic graph $\mathcal{G}$ (like the one in Fig. 3), the Gaussian BP algorithm is known to compute the correct marginal means μ_i and variances P_{ii} in at most d iterations, where d is the diameter of $\mathcal{G}$. For a general (cyclic) graph (like the one in Fig. 4), conditions under which the Gaussian BP algorithm converges have been studied extensively. [21] showed that under the assumption that J is diagonally dominant, $\mu_i(k)$ converges to μ_i asymptotically (as $k \rightarrow \infty$). This assumption was relaxed to generalised diagonal dominance in [23]. General, but more complex, conditions under which $\mu_i(k)$ converges correctly are given in [24,25]. Recall that a square matrix $A = \{a_{ij}\}$ is said to be *diagonally dominant* if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for all i, j , and *generalised diagonally dominant* if there exists a diagonal matrix $D > 0$ such that AD is diagonally dominant.

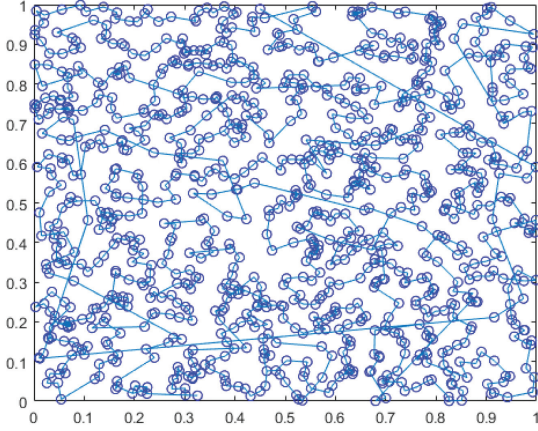


Fig. 4. A 1000-node loopy graph for Algorithm 3.

4. Distributed algorithm for Weighted Average Consensus (WAC)

An alternative formulation of the distributed average consensus is as follows:

$$\min_x \frac{1}{2} \sum_{i \in \mathcal{V}} w_i (x_i - u_i)^2$$

subject to $x_i = x_j, \forall (i, j) \in \mathcal{E}$. (16)

This is because it is straightforward to verify that, under the assumption that $\mathcal{G}$ is connected, the constraint above enforces that all x_i are identical, which in turn means that the optimal solution x^* to (16) is that $x_i^* = \bar{x}$ in (1) for all $i \in \mathcal{V}$.

Define the incidence matrix $B = \text{col}\{e_{ij} : (i, j) \in \mathcal{E}\}$, where $e_{ij} \in \mathbb{R}^n$ is a row vector which contains 1 in the i th element, -1 in j th element, and 0 elsewhere. Then, (16) is equivalent to

$$\min_x \frac{1}{2} (x - u)^T W (x - u), \text{ subject to } Bx = 0, \quad (17)$$

where $W = \text{diag}\{w_i\}$ and $u = \text{col}\{u_i\}$. Using the alternating direction method of multipliers (ADMM) method [26], (16) can be solved by considering the augmented function

$$L_\rho(x, y) = \frac{1}{2} (x - u)^T W (x - u) + y^T Bx + \frac{1}{2} \rho x^T Lx, \quad (18)$$

where $y \in \mathbb{R}^n$ is the Lagrange multiplier, $\rho > 0$ is the penalty parameter, and $L = B^T B = \{l_{ij}\}$ is the Laplacian matrix given by

$$l_{ij} = \begin{cases} |\mathcal{N}_i|, & j = i \\ -1, & j \neq i, j \in \mathcal{N}_i \\ 0, & j \neq i, j \notin \mathcal{N}_i \end{cases} \quad (19)$$

The solution is given by the iterative calculations below [26]:

$$x(t+1) = \arg \min_x L_\rho(x, y(t)), \quad (20)$$

$$y(t+1) = y(t) + \rho Bx(t) \quad (21)$$

with any initial $x(0)$ and $y(0)$, and $t = 0, 1, 2, \dots$. It is known [26] that correct (optimal) solution to (17) is produced by iterating (20)–(21).

It is clear that (20) is the key and most computationally demanding step. Solving (20) by setting $\partial L_\rho(x, y(t))/\partial x = 0$, which yields the following sparse linear system:

$$(W + \rho L)x = Wu - y(t). \quad (22)$$

The sparsity of the matrix $W + \rho L$ is due to the diagonal property of W and sparse connectivity of $\mathcal{G}$.

Algorithm 1 Distributed Algorithm for Distributed WAC [29]

• **Initialization:** For each node i , do: For each $j \in \mathcal{N}_i$, set $x_{i \rightarrow j}(0) = b_i, s_{i \rightarrow j}(0) = f(w_i)$ and transmit them to node j .

• **Main loop:** At iteration $k = 1, 2, \dots$, each node i computes

$$\tilde{s}_i(k) = w_i + \sum_{j \in \mathcal{N}_i} s_{j \rightarrow i}(k-1) \quad (23)$$

$$\tilde{x}_i(k) = w_i b_i + \sum_{j \in \mathcal{N}_i} s_{j \rightarrow i}(k-1) x_{j \rightarrow i}(k-1) \quad (24)$$

$$\hat{x}_i(k) = \frac{\tilde{x}_i(k)}{\tilde{s}_i(k)}, \quad (25)$$

then for each $j \in \mathcal{N}_i$, computes

$$s_{i \rightarrow j}(k) = f(\tilde{s}_i(k) - s_{j \rightarrow i}(k-1)) \quad (26)$$

$$x_{i \rightarrow j}(k) = \frac{\tilde{x}_i(k) - x_{j \rightarrow i}(k-1)}{\tilde{s}_i(k) - s_{j \rightarrow i}(k-1)} \quad (27)$$

and transmits them to node j .

The distributed algorithm for weighted average consensus is presented in Algorithm 3 below, which comes from [29]. For notational convenience, we denote $b = Wu - y(t)$ and $f(w) = \rho w / (\rho + w)$.

Algorithm 1 has the following key convergence properties. Please refer to [29] for proofs.

Theorem 1.

1. [29] Suppose the graph $\mathcal{G}$ is connected and acyclic with diameter d . Then, running Algorithm 1 yields the following finite convergence:

$$\hat{x}(k) = x^*, \quad \forall k \geq d \quad (28)$$

where $\hat{x}(k) = \text{col}\{\hat{x}_1(k), \hat{x}_2(k), \dots, \hat{x}_n(k)\}$.

2. [29] For a general connected (cyclic) graph, Algorithm 1 has the following exponential convergence:

$$\|\hat{x}_i(k) - x_i^*\| \leq \eta^{k-1} C, \quad \forall i \in \mathcal{V}, k = 0, 1, 2, \dots, \quad (29)$$

where $0 < \eta < 1$ is an upper bound on the convergence rate of (29), given by

$$\eta = \max_{i \in \mathcal{V}} \frac{(|\mathcal{N}_i| - 1)\rho}{w_i + (|\mathcal{N}_i| - 1)\rho}, \quad (30)$$

and C is a constant independent of k .

5. Distributed algorithm for WLS

It is straightforward to verify that the solution to the WLS problem in (5) is given by the following sparse linear system:

$$\Psi x = \alpha \quad (31)$$

with $\Psi = H^T R^{-1} H$ and $\alpha = H^T R^{-1} z$. Thus, our focus is to solve (31) in a distributed way by using the sparsity structure of H (or Ψ). Denote its solution by $x^* = \Psi^{-1} \alpha$.

A fully distributed WLS algorithm was presented in [8]. It is shown in Algorithm 2, where

$$\gamma_{ij} = B_{ij}^T S_{i,j}^{-1} B_{ji} \quad (32)$$

for $\forall i \in \mathcal{V}, j \in \mathcal{N}_i$. The estimate for x_i^* in iteration k is given by $\hat{x}_i(k)$ in (35).

The basic idea of Algorithm 2 is as follows. At initial iteration $k = 0$, each $i \in \mathcal{V}$ computes its local version of Ψ and α , i.e., $\hat{\Psi}_i(0)$ and $\hat{\alpha}_i(0)$,

respectively, by using all components related to x_i which are either self measurement or edge measurements. For each $j \in \mathcal{N}_i$, messages passed from i to j are defined by $\Psi_{i \rightarrow j}(0)$ and $\alpha_{i \rightarrow j}(0)$, which carry the neighbouring information of Ψ and α from node j to node i . At iteration $k > 0$, $\hat{\Psi}_i(k)$ and $\hat{\alpha}_i(k)$ can be updated using the initial value and messages sent by $j \in \mathcal{N}_i$ at the previous iteration, and the estimation $\hat{x}_i(k)$ can be expressed by $\hat{x}_i(k) = \hat{\Psi}_i(k)^{-1} \hat{\alpha}_i(k)$. Node i uses all messages sent by $v \in \mathcal{N}_i$ except j to calculate $\Psi_{i \rightarrow j}(k)$ and $\alpha_{i \rightarrow j}(k)$, and sends them to node j .

Algorithm 2 Distributed WLS Algorithm [8] (based on H)

- **Initialization:** Each node i computes

$$\check{\Psi}_i(0) = A_i^T R_i^{-1} A_i + \sum_{j \in \mathcal{N}_i} B_{ij}^T S_{ij}^{-1} B_{ij}$$

$$\check{\alpha}_i(0) = A_i^T R_i^{-1} z_{i,i} + \sum_{j \in \mathcal{N}_i} B_{ij}^T S_{ij}^{-1} z_{i,j}$$

and for each $j \in \mathcal{N}_i$

$$\Psi_{i \rightarrow j}(0) = \check{\Psi}_i(0)$$

$$\alpha_{i \rightarrow j}(0) = \check{\alpha}_i(0)$$

- **Main loop:** At $k = 0, 1, \dots$, each node i computes

$$\hat{\Psi}_i(k+1) = \check{\Psi}_i(0) - \sum_{v \in \mathcal{N}_i} \frac{\gamma_{vi}^T \gamma_{vi}}{\Psi_{v \rightarrow i}(k)} \quad (33)$$

$$\hat{\alpha}_i(k+1) = \check{\alpha}_i(0) - \sum_{v \in \mathcal{N}_i} \frac{\gamma_{vi}^T \alpha_{v \rightarrow i}(k)}{\Psi_{v \rightarrow i}(k)} \quad (34)$$

$$\hat{x}_i(k+1) = \frac{\hat{\alpha}_i(k+1)}{\hat{\Psi}_i(k+1)} \quad (35)$$

then for each $j \in \mathcal{N}_i$, computes

$$\Psi_{i \rightarrow j}(k+1) = \hat{\Psi}_i(k+1) + \frac{\gamma_{ji}^T \gamma_{ji}}{\Psi_{j \rightarrow i}(k)} \quad (36)$$

$$\alpha_{i \rightarrow j}(k+1) = \hat{\alpha}_i(k+1) + \frac{\gamma_{ji}^T \alpha_{j \rightarrow i}(k)}{\Psi_{j \rightarrow i}(k)} \quad (37)$$

and transmits them to node j .

The convergence properties of Algorithm 2 have been studied in [8, 30], and they are summarised below. Please refer to [8,30] for proofs.

Theorem 2.

1. [8] If $\mathcal{G}$ is connected and acyclic with diameter d , the estimate $\hat{x}_i(k)$ obtained by running Algorithm 2 converges to the exact value x_i^* in d iterations for all $i \in \mathcal{V}$.
2. [30] For a general connected (cyclic) graph and the types of measurements (2)–(3), the estimate $\hat{x}_i(k)$ obtained by running Algorithm 2 is asymptotically convergent to the exact value x_i^* as $k \rightarrow \infty$, for all $i \in \mathcal{V}$.
3. [30] The convergence rate is bounded as follows:

$$|\hat{x}_i(k) - x_i^*| \leq \rho(\bar{\Omega})^k C$$

for each $i \in \mathcal{V}$, where $\Omega = (\omega_{ij}) = I - D^{-1}\Psi$, $\bar{\Omega} = \{|\omega_{ij}|\}$, $D = \text{diag}\{\Psi_{ii}\}$, and $C > 0$ is a constant. In particular, $\rho(\bar{\Omega}) < 1$ is guaranteed.

6. Distributed solver for sparse linear systems

This section focuses on distributed solutions to a general sparse linear system (8). The key difference between (8) and sparse linear system (7) in the posterior probability problem that the latter has a symmetric and positive definite matrix J for which the Gaussian BP algorithm, which was developed specifically for Gaussian graphical models, applies.

It turns out that, under suitable conditions, the Gaussian BP algorithm can be generalised to handle an asymmetric matrix A in (8). The resulting distributed algorithm was developed in [27], as listed in Algorithm 3. In each iteration k of the algorithm, each node i computes variables $a_{i \rightarrow j}(k)$ and $b_{i \rightarrow j}(k)$ for each of its neighbouring node $j \in \mathcal{N}_i$ and transmit them to node j . All the nodes execute the same algorithm concurrently.

Algorithm 3 Distributed Solver for Linear systems [27]

- **Initialization:** For each node i , transmit $a_{i \rightarrow j}(0) = a_{ii}$ and $b_{i \rightarrow j}(0) = b_i$ to each node $j \in \mathcal{N}_i$.

- **Main loop:** At iteration $k = 1, 2, \dots$, each node i computes

$$\tilde{a}_i(k) = a_{ii} - \sum_{v \in \mathcal{N}_i} \frac{a_{vi} a_{iv}}{a_{v \rightarrow i}(k-1)} \quad (38)$$

$$\tilde{b}_i(k) = b_i - \sum_{v \in \mathcal{N}_i} \frac{a_{iv} b_{v \rightarrow i}(k-1)}{a_{v \rightarrow i}(k-1)} \quad (39)$$

$$\hat{x}_i(k) = \frac{\tilde{b}_i(k)}{\tilde{a}_i(k)}, \quad (40)$$

then for each $j \in \mathcal{N}_i$, computes

$$a_{i \rightarrow j}(k) = \tilde{a}_i(k) + \frac{a_{ji} a_{ij}}{a_{j \rightarrow i}(k-1)} \quad (41)$$

$$b_{i \rightarrow j}(k) = \tilde{b}_i(k) + \frac{a_{ij} b_{j \rightarrow i}(k-1)}{a_{j \rightarrow i}(k-1)} \quad (42)$$

and transmits them to node j .

Once again, we have the following convergence results. Please refer to [27,28,33] for proofs.

Theorem 3.

1. [27] For an acyclic graph $\mathcal{G}$ with diameter d , suppose the linear system (8) is generalised diagonally dominant. Then, running Algorithm 3 yields

$$\hat{x}_i(k) = x_i^*, \quad \forall k \geq d, i \in \mathcal{V}. \quad (43)$$

2. [27] For a general connected (cyclic) graph $\mathcal{G}$, suppose the linear system (8) is generalised diagonally dominant. Then, running Algorithm 3 yields

$$\lim_{k \rightarrow \infty} \hat{x}_i(k) = x_i^*, \quad \forall i \in \mathcal{V}. \quad (44)$$

3. ([28] and Corollary 1 of [33]) Moreover, defining the matrix $U = \{u_{ij}\}$ such that $u_{ii} = 0$ and $u_{ij} = |a_{ij}/a_{ii}|$ for all $i, j = 1, 2, \dots, n$ with $j \neq i$, then the convergence rate of $\hat{x}_i(k)$ is bounded by

$$|\hat{x}_i(k) - x_i^*| \leq C \rho(U)^k, \quad \forall i \in \mathcal{V}, k = 0, 1, 2, \dots, \quad (45)$$

where C is a constant and $\rho(U)$ is the spectral radius of U , with $\rho(U) < 1$ guaranteed when A is generalised diagonally dominant.

7. Distributed convex optimisation

In this section, we consider a more general distributed computation problem, namely, distributed convex optimisation problem. For a network graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node set $\mathcal{V} = \{1, 2, \dots, n\}$ and undirected

edge set $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$, we want to solve the distributed optimisation problem

$$\min_{x \in \mathbb{R}^n} F(x) \quad (46)$$

for a nonlinear objective function $F : \mathbb{R}^n \rightarrow \mathbb{R}$ having the following properties:

Assumption 1.

- $F(\cdot)$ is twice continuously differentiable, strictly convex and coercive;
- $F(\cdot)$ is expressed in a pairwise separation form (47) below with some graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$;
- $F(\cdot)$ is scaled diagonally dominant for some $0 < \lambda < 1$ and positive $w \in \mathbb{R}^n$.

where the notions of pairwise separation and scaled diagonal dominance are defined below.

Definition 1 (Pairwise Separation). An objective function $F : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be in a *pairwise separation* form if it is expressed as

$$F(x) = \sum_{i \in \mathcal{V}} f_i(x_i) + \sum_{(i,j) \in \mathcal{E}} f_{ij}(x_i, x_j), \quad (47)$$

where the factors $\{f_i(\cdot)\}$ and $\{f_{ij}(\cdot, \cdot)\}$ are twice continuously differentiable and coercive.

Definition 2 (Scaled Diagonal Dominance [31]). Given a twice continuously differentiable function $F : \mathbb{R}^n \rightarrow \mathbb{R}$, a scalar $\lambda \in (0, 1)$, and a positive vector $w = (w_1, w_2, \dots, w_n) \in \mathbb{R}^n$, then F (or, with some abuse of notation, $\frac{\partial^2}{\partial x^2} F(x)$) is said to be (λ, w) -scaled diagonally dominant if, for every $i \in \mathcal{V}$ and all $x \in \mathbb{R}^n$,

$$\sum_{j \in \mathcal{N}_i} w_j \left| \frac{\partial^2}{\partial x_i \partial x_j} F(x) \right| \leq \lambda w_i \frac{\partial^2}{\partial x_i^2} F(x). \quad (48)$$

Note that pairwise separation basically means that the overall objective function consists of local functions $f_i(x_i)$ and coupling functions $f_{ij}(x_i, x_j)$ between neighbouring nodes. This structure is natural for many applications. The scaled diagonal dominance basically means that the Hessian matrix of $F(x)$ is such that the local terms dominate the influence of the coupling terms in some scaled fashion. This is a sufficient condition for convexity of $F(x)$ and can be justified in many network applications.

The distributed algorithm we introduce is a variant of the *message passing* (or BP) algorithm. Denote the optimal solution of (46) by x^* . The algorithm is given below.

At iteration $k = 0, 1, \dots$, each node i takes an *incoming message* $J_{u \rightarrow i}^{(k)} : \mathbb{R} \rightarrow \mathbb{R}$ from each neighbouring node $u \in \mathcal{N}_i$. These incoming messages are fused together to create an *outgoing message* $J_{i \rightarrow j}^{(k+1)} : \mathbb{R} \rightarrow \mathbb{R}$ for each $j \in \mathcal{N}_i$ according to

$$J_{i \rightarrow j}^{(k+1)}(x_j) := \min_{y_i} f_i(y_i) + f_{ji}(x_j, y_i) + \sum_{u \in \mathcal{N}_i \setminus j} J_{u \rightarrow i}^{(k)}(y_i) + \kappa_{i \rightarrow j}^{(k+1)}, \quad (49)$$

where $\kappa_{i \rightarrow j}^{(k+1)}$ is such that $J_{i \rightarrow j}^{(k+1)}(0) = 0$. The local objective function for each x_i in iteration k is constructed as

$$\gamma_i^{(k)}(x_i) := f_i(x_i) + \sum_{u \in \mathcal{N}_i} J_{u \rightarrow i}^{(k)}(x_i), \quad (50)$$

and the estimate $x_i^{(k+1)}$ for x_i^* in iteration k is obtained by minimising $\gamma_i^{(k)}$, i.e.,

$$x_i^{(k+1)} := \arg\min \gamma_i^{(k)}(x_i). \quad (51)$$

We will consider the choice of initial messages as follows.

Assumption 2. Take any set of initial estimates $\{x_{i \rightarrow j}^{(0)}, i \in \mathcal{V}, j \in \mathcal{N}_i\}$ and any set of initial messages $\{J_{i \rightarrow j}^{(0)}(\cdot), i \in \mathcal{V}, j \in \mathcal{N}_i\}$ which are twice continuously differentiable and satisfy the following for some $0 \leq \rho < \lambda^{-1}$:

$$\begin{aligned} & \frac{d^2}{dx_j^2} J_{i \rightarrow j}^{(0)}(x_j) - \frac{d^2}{dx_j^2} f_{ji}(x_j, x_{i \rightarrow j}^{(0)}) \\ & \geq -\rho \frac{w_i}{w_j} \left| \frac{\partial^2}{\partial x_j \partial x_i} f_{ji}(x_j, x_{i \rightarrow j}^{(0)}) \right|, \quad \forall x_j \in \mathbb{R}. \end{aligned} \quad (52)$$

In particular, that any

$$J_{i \rightarrow j}^{(0)}(x_j) = f_{ji}(x_j, x_{i \rightarrow j}^{(0)}) + c_{ji}(x_j, x_{i \rightarrow j}^{(0)})$$

with $c_{ji}(\cdot, \cdot)$ being affine in the first variable will satisfy (52), including $c_{ji}(\cdot, \cdot) = 0$.

The main convergence properties of the message-passing algorithm above are given below; see [32] for details and proof.

Theorem 4. Under Assumptions 1–2, the following convergence property holds for every node $r \in \mathcal{V}$ and $k > 0$:

$$\begin{aligned} & \frac{|x_r^{(k)} - x_r^*|}{w_r} \\ & \leq \frac{\lambda^k}{1 - \lambda} \max_{i \in \mathcal{V}} \frac{\sum_{u \in \mathcal{N}_i} |\nabla f_{iu}(x_i^*, x_u^*) - \nabla J_{u \rightarrow i}^{(0)}(x_i^*)|}{w_i \min_{x \in \mathbb{R}^n} \frac{\partial^2}{\partial x_i^2} F(x)}. \end{aligned} \quad (53)$$

Moreover, if the initial messages are chosen to be $J_{i \rightarrow j}^{(0)}(x_j) = f_{ji}(x_j, x_i^{(0)})$ and $x_{i \rightarrow j}^{(0)} = x_i^{(0)}$ for all $i \in \mathcal{V}, j \in \mathcal{N}_i$, then (53) simplifies to

$$\frac{|x_r^{(k)} - x_r^*|}{w_r} \leq \lambda^{k+1} \frac{M}{1 - \lambda} \max_{v \in \mathcal{V}} \frac{|x_v^{(0)} - x_v^*|}{w_v}, \quad (54)$$

where M is the conditioning value for the diagonal part of the Jacobian matrix for $F(\cdot)$, i.e.,

$$M = \max_{i \in \mathcal{V}} \frac{\max_{x \in \mathbb{R}^n} \frac{\partial^2}{\partial x_i^2} F(x)}{\min_{x \in \mathbb{R}^n} \frac{\partial^2}{\partial x_i^2} F(x)}. \quad (55)$$

We see from (54) that, with a simple choice of initial messages, the estimate $x^{(k)}$ will converge to the optimal solution x^* exponentially fast. This superb convergence property is achieved with the simple assumptions of pairwise separation and scaled diagonal dominance. Since these assumptions can be satisfied in many applications, the message-passing algorithm is indeed a powerful distributed convex optimisation tool.

8. Examples

In this section, we provide some examples to illustrate the distributed algorithms introduced in this paper.

8.1. Example 1: Distributed weighted average consensus

To demonstrate the performance of Algorithm 1 on loopy graphs, we apply it to the 13-node graph in Fig. 5, with $w_i = 1, y_i = i, i = 1, 2, \dots, 13$, and $\gamma = 1000$. The weighted average consensus (1) is easily checked to give $\bar{x} = 7$. The result of Algorithm 1 is given in Fig. 6. It can be verified by analysing Fig. 6 that the convergence rate of $\|\hat{x}^{(k)} - x^*\|$ turns out to be much faster than the estimate in (29)–(30).

We also note that applications of Algorithm 1 in distributed Kalman filtering and distributed clock synchronisation can be found in [2,3], respectively.

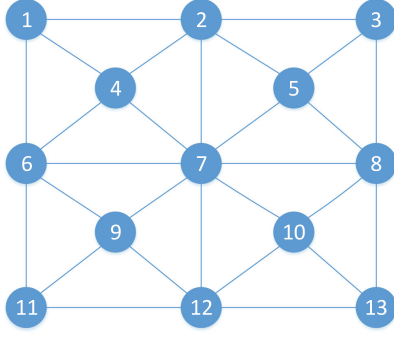


Fig. 5. A 13-node loopy graph.

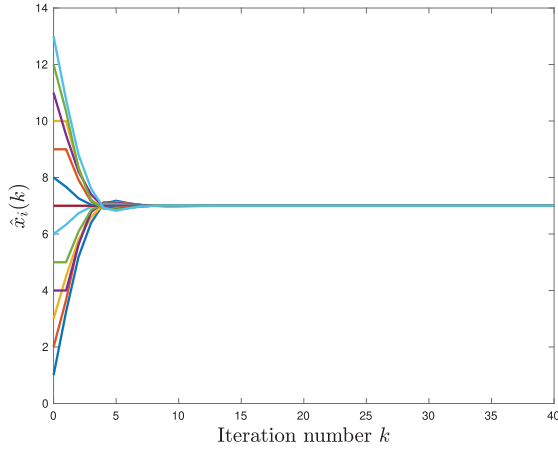


Fig. 6. Convergence of the proposed Algorithm 1.

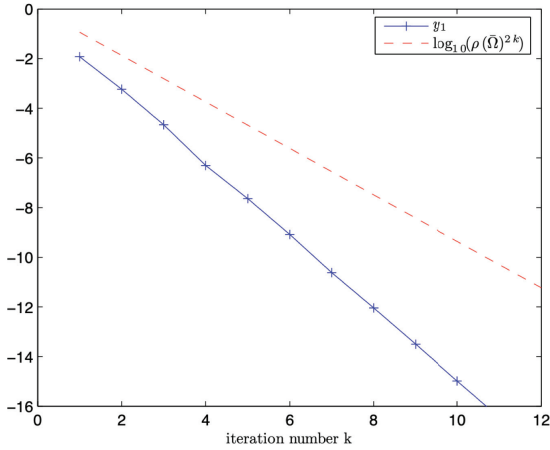


Fig. 7. WLS estimation error for the 13 node network.

8.2. Example 2: Distributed WLS

We now study the loopy network with 13 nodes in Fig. 5 again. In this network, there are two nodes without self measurement, i.e., for $i \in \{2, 5\}$, $A_i = 0$. The other nonzero A_i and all the B_{ij} are chosen randomly. Fig. 7 shows the estimation error by Algorithm 2 using $y_1 = \log_{10}\{\sum_i (\hat{x}_i(k) - x_i^*)^2/n\}$. We see from Fig. 7 that the estimation error converges to zero exponentially fast as the iteration number increases.

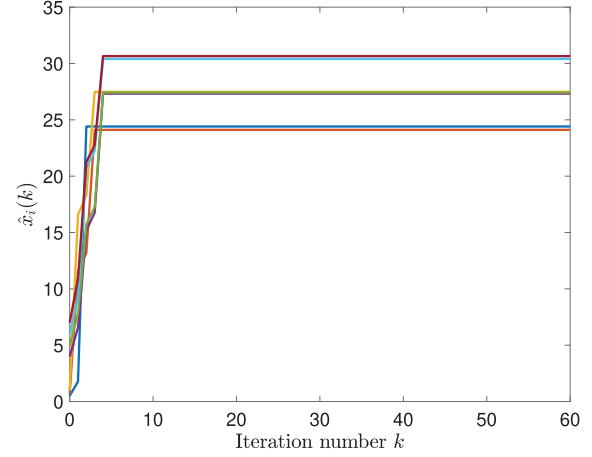


Fig. 8. Convergence of Algorithm 3 on acyclic graph.

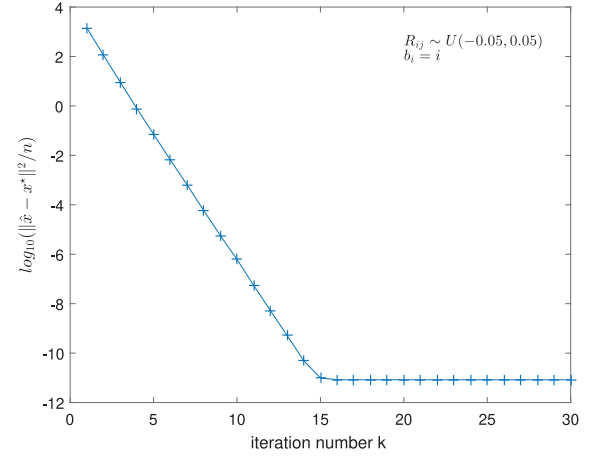


Fig. 9. Convergence error of the proposed Algorithm 3.

8.3. Example 3: Distributed linear system solver

We first demonstrate Algorithm 3 on an acyclic graph depicted in Fig. 3. The matrix A has $a_{ii} = |\mathcal{N}_i|$ and the non-zero a_{ij} randomly chosen from $(-1, -0.85)$ and $b_i = i$. It is easy to check that the diameter of the graph is $d = 4$. The simulated result for Algorithm 3 is shown in Fig. 8. As we can see, it takes only 4 iterations to converge.

Next, we test Algorithm 3 on a randomly connected loopy graph with 1000 nodes, as shown in Fig. 4. The circles represent the nodes and the curves represent the edges. The matrix A is selected by taking the non-zero off-diagonal terms R_{ij} (the (i, j) th entry of R) to be random within $(-0.05, 0.05)$, and $b_i = i$ for all i . We plot the logarithmic error $\log_{10}(\|\hat{x}(k) - x^*\|^2/n)$ versus the iteration number k in Fig. 9. As we can see from the figure, the error also decreases exponentially (or the logarithmic error decreases linearly) as the iteration number increases. The error floor after 15 iterations is due to the accuracy limitation of Matlab.

8.4. Example 4: Distributed convex optimisation

To illustrate the distributed algorithm for convex optimisation, we revisit the 13-node graph in Fig. 5 and consider the objective function $F(x)$ with

$$f_i(x_i) = \frac{1}{2}a_{ii}x_i^2 - b_ix_i + \phi(x_i),$$

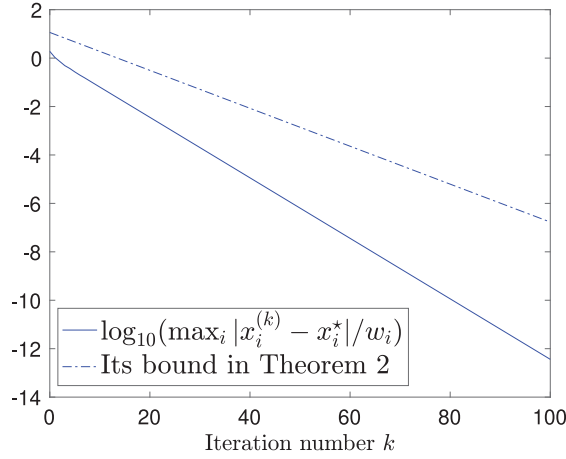


Fig. 10. Convergence of the 13-node system in Example 4.

$$f_{ij}(x_i, x_j) = a_{ij}x_i x_j, \quad (56)$$

where $a_{ii} = |\mathcal{N}_i|$, the non-zero $a_{ij} = a_{ji}$ are chosen uniformly from $(-1.2, -0.2)$, and b_i are chosen randomly using the normal distribution $\mathcal{N}(0, 1)$. The function $\phi(x_i)$ is a barrier function given by

$$\phi(x_i) = \begin{cases} -\epsilon \log(d^2 - x_i^2), & |x_i| < d \\ \infty, & \text{otherwise} \end{cases} \quad (57)$$

for some given $d > 0$ and some (sufficiently) small $\epsilon > 0$. This is used to relax a hard constraint of $|x_i| < d$. That is, for an interior point of $|x_i| < d$, the barrier function becomes void, but it becomes dominant as x_i moves towards the boundary of $|x_i| = d$. When the barrier function is neglected, the scaled diagonal dominance property can be simply checked with $\lambda \approx 0.835$.

We initialise $x_i^{(0)} = \text{sat}(b_i/(a_{ii}d))d$ and $J_{i \rightarrow j}^{(0)}(x_j) = a_{ji}x_j x_i^{(0)}$ for all i, j , where $\text{sat}(\cdot)$ is the saturation function with saturation level of 1. We have the exponential convergence result in Fig. 10, as predicted by Theorem 4.

9. Conclusion

We have introduced a number of distributed algorithms for solving problems in large scale networked systems. These algorithms are efficient and meet the complexity constraints specified in Section 2.5. These algorithms are related, and all evolved from the well-known BP algorithm, but tailored for different applications. Like the BP algorithm, we expect the distributed algorithms introduced here to find wide applications in sensor fusion, distributed estimation, distributed optimisation, distributed control, and distributed learning. Future research can be directed to asynchronous implementation of these algorithms, as well as more study of their convergence properties and generalisation to distributed nonlinear estimation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] S. Kar, J.M.F. Moura, K. Ramanan, Distributed parameter estimation in sensor networks: Nonlinear observation models and imperfect communication, *IEEE Trans. Inform. Theory* 58 (6) (2012) 1–52.
- [2] Z. Wu, M. Fu, Y. Xu, R. Lu, A distributed Kalman filtering algorithm with fast finite-time convergence for sensor networks, *Automatica* 95 (2018) 63–72.
- [3] K. Xie, Q. Cai, M. Fu, A fast clock synchronization algorithm for wireless sensor networks, *Automatica* 92 (2018) 133–142.
- [4] S. Mou, J. Liu, A.S. Morse, A distributed algorithm for solving a linear algebraic equation, *IEEE Trans. Automat. Control* 60 (11) (2015) 2863–2878.
- [5] S. Mou, J. Liu, A.S. Morse, Asynchronous distributed algorithms for solving linear algebraic equations, *IEEE Trans. Automat. Control* 63 (2) (2018) 372–385.
- [6] X. Wang, S. Mou, Improvement of a distributed algorithm for solving linear equations, *IEEE Trans. Ind. Electron.* 64 (4) (2017) 3113–3117.
- [7] L. Xiao, S. Boyd, Fast linear iterations for distributed averaging, *Systems Control Lett.* 53 (2004) 65–78.
- [8] D. Marelli, M. Fu, Distributed weighted least-squares estimation with fast convergence for large-scale systems, *Automatica* 51 (2015) 27–39.
- [9] W.J. Russell, D.J. Klein, J.P. Hespanha, Optimal estimation on the graph cycle space, *IEEE Trans. Signal Process.* 59 (6) (2011) 2834–2846.
- [10] A. Bertrand, M. Moonen, Consensus-based distributed total least squares estimation in ad hoc wireless sensor networks, *IEEE Trans. Signal Process.* 59 (5) (2011) 2320–2330.
- [11] A. Bertrand, M. Moonen, Low-complexity distributed total least squares estimation in ad hoc sensor networks, *IEEE Trans. Signal Process.* 60 (8) (2012) 4321–4333.
- [12] M. Nagy, Z. Akos, D. Biro, T. Vicsek, Hierarchical group dynamics in pigeon flocks, *Nature* 464 (2010) 890–893.
- [13] T. Nepusz, T. Vicsek, Controlling edge dynamics in complex networks, *Nat. Phys.* (2012) Advance Online Publication.
- [14] N. Tarczi, et al., Patterns, transitions and the role of leaders in the collective dynamics of a simple robotic flock, *J. Stat. Mech. Theory Exp.* (4) (2011) P04010.
- [15] Z. Lin, L. Wang, Z. Han, M. Fu, Distributed formation control of multi-agent systems using complex Laplacian, *IEEE Trans. Automat. Control* 59 (7) (2014) 1765–1777.
- [16] Z. Lin, L. Wang, Z. Han, M. Fu, A graph Laplacian approach to coordinate-free formation stabilization for directed networks, *IEEE Trans. Automat. Control* 61 (5) (2016) 1269–1280.
- [17] A. Nedic, A. Ozdaglar, Distributed sub-gradient methods for multi-agent optimization, *IEEE Trans. Automat. Control* 54 (1) (2009) 48–61.
- [18] D. Jakovetic, J.M.F. Moura, J. Xavier, Fast distributed gradient methods, *IEEE Trans. Automat. Control* 59 (5) (2014) 1131–1146.
- [19] J. Lu, C.Y. Tang, A distributed algorithm for solving positive definite linear equations over networks with membership dynamics, *IEEE Trans. Control Netw. Syst.* 5 (1) (2018) 215–227.
- [20] J. Pearl, *Probabilistic Reasoning in Intelligent Systems*, Morgan Kaufman, 1988.
- [21] Y. Weiss, William T. Freeman, Correctness of belief propagation in Gaussian graphical models of arbitrary topology, *Neural Comput.* 13 (10) (2001) 2173–2200.
- [22] R. Shental, et al., Gaussian belief propagation solver for systems of linear equations, in: *ISIT 2008*, Toronto, Canada, 2008, pp. 6–11.
- [23] D.M. Malioutov, J.K. Johnson, A.S. Willsky, Walk-sums and belief propagation in Gaussian graphical models, *J. Mach. Learn. Res.* 7 (2006) 2031–2064.
- [24] Q. Su, Y.-C. Wu, Convergence analysis of the variance in Gaussian belief propagation, *IEEE Trans. Signal Process.* 62 (19) (2014) 5119–5131.
- [25] Q. Su, Y.-C. Wu, Convergence conditions of Gaussian belief propagation, *IEEE Trans. Signal Process.* 63 (5) (2015) 1144–1155.
- [26] S. Boyd, et al., Distributed optimisation and statistical learning via the alternating direction method of multipliers, *Found. Trends Mach. Learn.* 3 (1) (2010) 1–122.
- [27] Z. Zhang, Q. Cai, M. Fu, A fast distributed solver for linear systems under generalized diagonal dominance, *IEEE Trans. Automat. Control* 66 (5) (2021) 2423–2429.
- [28] Z. Zhang, M. Fu, Convergence rate analysis of Gaussian belief propagation for Markov networks, *IEEE/CAA J. Autom. Sin.* 7 (3) (2020) 668–673.
- [29] K. Xie, Q. Cai, Z. Zhang, M. Fu, Distributed algorithms for average consensus of input data with fast convergence, *IEEE Trans. Syst. Man Cybern.: Syst.* 51 (5) (2021) 2653–2664.
- [30] Q. Yang, Z. Zhang, M. Fu, Distributed weighted least-squares estimation for networked systems with edge measurements, *Automatica* 120 (109091) (2020).
- [31] C.C. Moallemi, B. Van Roy, Convergence of min-sum message-passing for convex optimization, *IEEE Trans. Inform. Theory* 56 (4) (2010) 2041–2050.
- [32] Z. Zhang, M. Fu, Convergence properties of message-passing algorithm for distributed convex optimisation with scaled diagonal dominance, *IEEE Trans. Signal Process.* 69 (1) (2021) 3868–3877.
- [33] Z. Zhang, Q. Cai, M. Fu, Convergence rate of a message-passing algorithm for solving linear systems, *arXiv:2004.06327*.



Qianqian Cai received the B.S. degree in environmental engineering from the University of Shanghai for Science and Technology, Shanghai, China, in 2011, and the Ph.D. degree in engineering from the University of Newcastle, Callaghan, NSW, Australia, in 2016. She is currently an Associate Professor with the School of Automation, Guangdong University of Technology, Guangdong, China. Her current research interests include water pollution control engineering, environmental monitoring, reinforcement learning, networked control system and state estimation.



Zhaorong Zhang received her Bachelor's degree of engineering from Nanjing University of Science and Technology, Nanjing, China, in 2016 and Ph.D. degree from University of Newcastle, Newcastle, Australia, in 2021. She is currently a postdoctoral fellow in the Department of Applied Mathematics, Hong Kong Polytechnic University. Her recent research interests include distributed algorithms, reinforcement learning and networked control systems.



Minyue Fu received his Bachelor's Degree in Electrical Engineering from the University of Science and Technology of China, Hefei, China, in 1982, and Master and Ph.D. degrees in Electrical Engineering from University of Wisconsin-Madison, USA in 1983 and 1987, respectively. From 1987 to 1989, he served as an Assistant Professor in the Department of Electrical and Computer Engineering, Wayne State University, USA. He joined the University of Newcastle, Australia, in 1989 and became a Chair Professor in Electrical Engineering in 2003. In 2023, he joined the Southern University of Science and Technology, Shenzhen, China. His current research interests include networked control systems, distributed estimation and control, high-precision control, and reinforcement learning. He has been an Associate Editor for the IEEE Transactions on Automatic Control, IEEE Transactions on Signal Processing, Automatica and Journal of Optimisation and Engineering. He is a Fellow of IEEE, Fellow of IFAC, Fellow of Engineers Australia, and Fellow of Chinese Association of Automation.